```
import numpy as np
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# 1. GENERATE DUMMY DOG/CAT DATASET
# Imagine 100 animals, 10 total features (only 4 are actually useful to distinguish dog vs cat)
X_data, y_data = make_classification(
    n_samples=100,
    n_features=10,
    n_informative=4,
    n_classes=2,
    random_state=42
)
X_train, X_test, y_train, y_test = train_test_split(X_data, y_data, test_size=0.3, random_state=42)

# 2. SETUP PARAMETERS
swarm_size = 20
iterations = 50
w = 0.5
c1 = 1.5
c2 = 1.5
num_features = X_data.shape[1]

# Sigmoid function to convert continuous velocity to binary probability
def sigmoid(v):
    return 1 / (1 + np.exp(-np.clip(v, -10, 10)))

# FITNESS FUNCTION: Evaluates how good a feature subset is
def calculate_fitness(position):
    # If no features are selected, give it worst fitness
    if np.sum(position) == 0:
        return 0.0

    # Extract only the selected features (where position == 1)
    X_train_selected = X_train[:, position == 1]
    X_test_selected = X_test[:, position == 1]

    # Train a simple classifier
    clf = KNeighborsClassifier(n_neighbors=3)
    clf.fit(X_train_selected, y_train)

    # Test accuracy
    predictions = clf.predict(X_test_selected)
    return accuracy_score(y_test, predictions)

# 3. INITIALIZE SWARM
# Positions (binary mask: 0 or 1 for each feature)
positions = np.random.randint(2, size=(swarm_size, num_features))
# Velocities (continuous values)
velocities = np.random.uniform(-1, 1, size=(swarm_size, num_features))

# Personal bests
p_best_positions = np.copy(positions)
p_best_fitness = np.array([calculate_fitness(p) for p in positions])

# Global best
g_best_index = np.argmax(p_best_fitness)
g_best_position = np.copy(p_best_positions[g_best_index])
g_best_fitness = p_best_fitness[g_best_index]

# 4. START OPTIMIZATION LOOP
for it in range(iterations):
    for i in range(swarm_size):
        r1 = np.random.rand(num_features)
        r2 = np.random.rand(num_features)

        # Update Velocity
        velocities[i] = (w * velocities[i]) + \
                        (c1 * r1 * (p_best_positions[i] - positions[i])) + \
                        (c2 * r2 * (g_best_position - positions[i]))

        # Update Position using Sigmoid (Binary PSO transformation)
        s_v = sigmoid(velocities[i])
```

```
        positions[i] = np.where(np.random.rand(num_features) < s_v, 1, 0)

        # Calculate New Fitness
        fitness = calculate_fitness(positions[i])

        # Update Personal Best
        if fitness > p_best_fitness[i]:
            p_best_positions[i] = np.copy(positions[i])
            p_best_fitness[i] = fitness

        # Update Global Best
        if fitness > g_best_fitness:
            g_best_position = np.copy(positions[i])
            g_best_fitness = fitness

    print(f"Iteration {it+1}/{iterations} - Best Accuracy: {g_best_fitness:.4f}")

# 5. PRINT FINAL RESULT
print("\n--- OPTIMIZATION COMPLETE ---")
print("Best Feature Mask (1 = Keep, 0 = Drop):", g_best_position)
print(f"Indices of Selected Features: {np.where(g_best_position == 1)[0]}")
print(f"Final Classification Accuracy: {g_best_fitness * 100:.2f}%")
```

```
Iteration 1/50 - Best Accuracy: 0.8333
Iteration 2/50 - Best Accuracy: 0.8667
Iteration 3/50 - Best Accuracy: 0.8667
Iteration 4/50 - Best Accuracy: 0.8667
Iteration 5/50 - Best Accuracy: 0.8667
Iteration 6/50 - Best Accuracy: 0.8667
Iteration 7/50 - Best Accuracy: 0.8667
Iteration 8/50 - Best Accuracy: 0.8667
Iteration 9/50 - Best Accuracy: 0.9000
Iteration 10/50 - Best Accuracy: 0.9000
Iteration 11/50 - Best Accuracy: 0.9000
Iteration 12/50 - Best Accuracy: 0.9000
Iteration 13/50 - Best Accuracy: 0.9000
Iteration 14/50 - Best Accuracy: 0.9000
Iteration 15/50 - Best Accuracy: 0.9000
Iteration 16/50 - Best Accuracy: 0.9000
Iteration 17/50 - Best Accuracy: 0.9000
Iteration 18/50 - Best Accuracy: 0.9000
Iteration 19/50 - Best Accuracy: 0.9000
Iteration 20/50 - Best Accuracy: 0.9000
Iteration 21/50 - Best Accuracy: 0.9000
Iteration 22/50 - Best Accuracy: 0.9000
Iteration 23/50 - Best Accuracy: 0.9000
Iteration 24/50 - Best Accuracy: 0.9000
Iteration 25/50 - Best Accuracy: 0.9000
Iteration 26/50 - Best Accuracy: 0.9000
Iteration 27/50 - Best Accuracy: 0.9000
Iteration 28/50 - Best Accuracy: 0.9000
Iteration 29/50 - Best Accuracy: 0.9000
Iteration 30/50 - Best Accuracy: 0.9000
Iteration 31/50 - Best Accuracy: 0.9000
Iteration 32/50 - Best Accuracy: 0.9000
Iteration 33/50 - Best Accuracy: 0.9000
Iteration 34/50 - Best Accuracy: 0.9000
Iteration 35/50 - Best Accuracy: 0.9000
Iteration 36/50 - Best Accuracy: 0.9000
Iteration 37/50 - Best Accuracy: 0.9000
Iteration 38/50 - Best Accuracy: 0.9000
Iteration 39/50 - Best Accuracy: 0.9000
Iteration 40/50 - Best Accuracy: 0.9000
Iteration 41/50 - Best Accuracy: 0.9000
Iteration 42/50 - Best Accuracy: 0.9000
Iteration 43/50 - Best Accuracy: 0.9000
Iteration 44/50 - Best Accuracy: 0.9000
Iteration 45/50 - Best Accuracy: 0.9000
Iteration 46/50 - Best Accuracy: 0.9000
Iteration 47/50 - Best Accuracy: 0.9000
Iteration 48/50 - Best Accuracy: 0.9000
Iteration 49/50 - Best Accuracy: 0.9000
Iteration 50/50 - Best Accuracy: 0.9000

--- OPTIMIZATION COMPLETE ---
Best Feature Mask (1 = Keep, 0 = Drop): [1 0 1 1 0 1 0 0 0 1]
Indices of Selected Features: [0 2 3 5 9]
Final Classification Accuracy: 90.00%
```